

Coffer MCP — Architecture and Security Review

Version: 0.2.0 | **Author:** Anna Hix | **Date:** March 19, 2026 **Repository:** github.com/annawhooo/coffer-mcp |

Classification: Internal — For Security Review

1. Executive Summary

Coffer is an open-source Python MCP (Model Context Protocol) server that provides encrypted credential storage and authenticated request proxying for LLM agents. It enables AI assistants like Claude to make authenticated HTTP requests and access login-protected web portals without the credential ever appearing in the LLM's conversation context.

Key security properties:

- Credentials encrypted at rest with AES-256-GCM (per-entry unique nonces)
- Master key stored in OS keyring with random per-user PBKDF2 salt
- URL allowlisting with strict domain matching prevents credential use against unauthorized endpoints (fail-closed)
- Per-hop redirect checking against URL allowlist prevents open-redirect credential theft
- Response sanitization scrubs leaked credentials from bodies and error messages
- Prompt injection defense strips HTML comments, hidden elements, and invisible unicode from responses
- Browser sessions auto-expire after 30 minutes with URL allowlist enforcement on every navigation
- HMAC-SHA-256 audit chain keyed to master key (file-only attackers cannot forge entries)
- Credential expiry with automatic enforcement and expiring-soon warnings
- OAuth2 client_credentials with automatic token refresh
- Encrypted backup/restore for disaster recovery
- Credential health checks (coffer test) to verify credentials before use
- Credentials never cross into the LLM context window

2. Architecture Overview

2.1 Trust Boundaries

The system operates across four trust boundaries:

Boundary 1 — User-controlled (one-time setup). The user interacts with the Coffey CLI to initialize the master key and add credentials. Plaintext credentials exist only during this interactive session (masked input via `getpass`). The master key is derived via PBKDF2-HMAC-SHA256 (600,000 iterations, random 16-byte salt) and stored in the OS keyring.

Boundary 2 — Coffey MCP server process (localhost, STDIO). The MCP server runs as a local subprocess spawned by Claude Desktop. It communicates via STDIO (stdin/stdout JSON-RPC). All credential resolution, HTTP request injection, browser automation, and response sanitization happen inside this process. The server never listens on a network port.

Boundary 3 — LLM context (Claude Desktop). Claude Desktop communicates with Coffey via MCP tool calls over STDIO. Claude sends tool invocations (alias, URL, method) and receives sanitized results. Claude never receives passwords, API keys, tokens, session cookies, or usernames.

Boundary 4 — External targets (REST APIs, web portals). Authenticated requests are made from the Coffey server process to external targets. Credentials are injected into HTTP headers (for API proxy) or browser form fields (for web login). TLS handles in-transit encryption. HTTP redirects are checked per-hop against the URL allowlist before following.

2.2 Components

Component	Role	Technology
Encrypted store	Per-entry AES-256-GCM encryption	Python cryptography library
Keychain	Master key from OS keyring (random salt)	Python keyring library
URL allowlist	Strict domain + path enforcement	urlparse + fnmatch (path only)
Response sanitizer	Scrubs leaked creds from responses	String + base64 + URL-encoding
Content sanitizer	Strips prompt injection payloads	Regex (HTML comments, hidden elements, invisible unicode)
Audit logger	Append-only JSONL, HMAC-SHA-256 chain	JSON + hmac + hashlib
HTTP proxy tool	Injects auth headers, checks redirects	httpx (async, no auto-follow)
OAuth2 token mgr	Auto-fetch/cache/refresh OAuth2 tokens	httpx + in-memory cache
Credential test tool	Health check (pass/fail + latency)	httpx HEAD request
Web login tool	Headless browser form automation	Playwright (Chromium)

Web fetch tool	Authenticated content as markdown	Playwright + readability-lxml
Backup/restore	Encrypted export/import	AES-256-GCM with separate passphrase
MCP server	Registers tools, manages lifecycle	FastMCP over STDIO
CLI	Credential mgmt outside of Claude	Click